Machine learning Lab Observation

ITA0614 - Machine Learning

Experiment 3

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Name : Variganji Sumanth Babu

Reg.No : 192524447

Problem 1: Play Tennis

Aim

To implement the ID3 decision-tree algorithm on the classic Play Tennis data and classify a new weather sample.

Algorithm

1. Calculate the entropy of the target class in the training data.
2. Calculate information gain for every available attribute and select the attribute with the highest gain.
3. Create a decision node for that attribute and split the records by its categorical values.
4. Repeat ID3 on each subset using the remaining attributes; stop when a subset is pure or no attributes remain.
5. Follow the learned branches to classify the new sample; use the node majority for an unseen branch.

Entropy(S) = -Σ $p_i \log_2 p_i$; Gain(S, A) = Entropy(S) - weighted entropy after splitting on A.

Program:

id3_a.py - Python 3

```python
from collections import Counter
from math import log2


def entropy(labels):
    c = Counter(labels); n = len(labels)
    return -sum(x / n * log2(x / n) for x in c.values())

def gain(rows, attr, target):
    n = len(rows); base = entropy([r[target] for r in rows]); rest = 0.0
    for v in {r[attr] for r in rows}:
        part = [r for r in rows if r[attr] == v]
        rest += len(part) / n * entropy([r[target] for r in part])
    return base - rest

def id3(rows, attrs, target):
    labels = {r[target] for r in rows}
    if len(labels) == 1: return {"label": next(iter(labels))}
    if not attrs: return {"label": Counter(r[target] for r in rows).most_common(1)[0][0]}
    scores = {a: gain(rows, a, target) for a in attrs}; best = max(attrs, key=scores.get)
    node = {"attribute": best, "default": Counter(r[target] for r in rows).most_common(1)[0][0], "branches": {}}
    for v in sorted({r[best] for r in rows}):
        part = [r for r in rows if r[best] == v]
        node["branches"][v] = id3(part, [a for a in attrs if a != best], target)
    return node

def predict(node, sample):
    if "label" in node: return node["label"]
    child = node["branches"].get(sample.get(node["attribute"]))
    return predict(child, sample) if child else node["default"]

def show_tree(node, indent=""):
    if "label" in node:
        print(indent + "-> " + node["label"]); return
    print(indent + node["attribute"])
    for value, child in sorted(node["branches"].items()):
        if "label" in child: print(indent + "  " + value + " -> " + child["label"])
        else:
            print(indent + "  " + value + " ->")
            show_tree(child, indent + "    ")

headers = ['Outlook', 'Temperature', 'Humidity', 'Wind', 'Play']
raw_rows = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No'],
]
records = [dict(zip(headers, row)) for row in raw_rows]
attributes = ['Outlook', 'Temperature', 'Humidity', 'Wind']
target = 'Play'
sample = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong',
}
tree = id3(records, attributes, target)
print("Training examples:", len(records))
print("Root attribute:", tree.get("attribute", tree.get("label")))
print("Decision tree:"); show_tree(tree)
print("New sample:", sample)
print("Prediction:", predict(tree, sample))
```

Input

Play Tennis data set: 14 training examples, 4 attributes (Outlook, Temperature, Humidity, Wind) and target concept Play (Yes / No). The categorical examples are stored in a Python list.

Outlook	Temperature	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Output

Terminal - Output

```
$ python id3_a.py
Training examples: 14
Root attribute: Outlook
Decision tree:
Outlook
  Overcast -> Yes
  Rain ->
    Wind
      Strong -> No
      Weak -> Yes
  Sunny ->
    Humidity
      High -> No
      Normal -> Yes
New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Prediction: No
```

Result

The ID3 tree selects Outlook as the root attribute. For the new sample, Outlook = Sunny leads to the Humidity branch; High humidity predicts Play = No. The tree is learned from the supplied categorical examples.

Problem 2: Student Result

Aim

To implement ID3 using CSV-formatted student records and predict whether a new student passes or fails.

Algorithm

1. Calculate the entropy of the target class in the training data.
2. Calculate information gain for every available attribute and select the attribute with the highest gain.
3. Create a decision node for that attribute and split the records by its categorical values.
4. Repeat ID3 on each subset using the remaining attributes; stop when a subset is pure or no attributes remain.
5. Follow the learned branches to classify the new sample; use the node majority for an unseen branch.

Entropy(S) = $-\Sigma\, p_i \log_2 p_i$; Gain(S, A) = Entropy(S) - weighted entropy after splitting on A.

Program:

id3_b.py - Python 3

```python
from collections import Counter
from math import log2
import csv
from io import StringIO

def entropy(labels):
    c = Counter(labels); n = len(labels)
    return -sum(x / n * log2(x / n) for x in c.values())

def gain(rows, attr, target):
    n = len(rows); base = entropy([r[target] for r in rows]); rest = 0.0
    for v in {r[attr] for r in rows}:
        part = [r for r in rows if r[attr] == v]
        rest += len(part) / n * entropy([r[target] for r in part])
    return base - rest

def id3(rows, attrs, target):
    labels = {r[target] for r in rows}
    if len(labels) == 1: return {"label": next(iter(labels))}
    if not attrs: return {"label": Counter(r[target] for r in rows).most_common(1)[0][0]}
    scores = {a: gain(rows, a, target) for a in attrs}; best = max(attrs, key=scores.get)
    node = {"attribute": best, "default": Counter(r[target] for r in rows).most_common(1)[0][0], "branches": {}}
    for v in sorted({r[best] for r in rows}):
        part = [r for r in rows if r[best] == v]
        node["branches"][v] = id3(part, [a for a in attrs if a != best], target)
    return node

def predict(node, sample):
    if "label" in node: return node["label"]
    child = node["branches"].get(sample.get(node["attribute"]))
    return predict(child, sample) if child else node["default"]

def show_tree(node, indent=""):
    if "label" in node:
        print(indent + "-> " + node["label"]); return
    print(indent + node["attribute"])
    for value, child in sorted(node["branches"].items()):
        if "label" in child: print(indent + "  " + value + " -> " + child["label"])
        else:
            print(indent + "  " + value + " ->")
            show_tree(child, indent + "    ")

csv_text = """
Study,Attendance,Assignment,Result
High,High,Done,Pass
High,High,Pending,Pass
High,Low,Done,Pass
Medium,High,Done,Pass
Medium,Low,Done,Fail
Low,High,Done,Fail
Low,Low,Pending,Fail
Low,Low,Done,Fail
"""
records = list(csv.DictReader(StringIO(csv_text.strip())))
attributes = ['Study', 'Attendance', 'Assignment']
target = 'Result'
sample = {
    'Study': 'Medium',
    'Attendance': 'Low',
    'Assignment': 'Pending',
}
tree = id3(records, attributes, target)
print("Training examples:", len(records))
print("Root attribute:", tree.get("attribute", tree.get("label")))
print("Decision tree:"); show_tree(tree)
print("New sample:", sample)
print("Prediction:", predict(tree, sample))
```

Input

Student Result data set: 8 illustrative examples, 3 attributes (Study, Attendance, Assignment) and target Result (Pass / Fail). The records are read from an embedded CSV stream with Python csv.DictReader.

Study	Attendance	Assignment	Result
High	High	Done	Pass
High	High	Pending	Pass
High	Low	Done	Pass
Medium	High	Done	Pass
Medium	Low	Done	Fail
Low	High	Done	Fail
Low	Low	Pending	Fail
Low	Low	Done	Fail

Output



Result

The ID3 tree uses Study as the root attribute for this illustrative data. For the new student with Medium study and Low attendance, the learned tree predicts Result = Fail.

Problem 3: Bank Loan Approval

Aim

To build an ID3 decision tree from a pandas DataFrame and use it to classify a new loan application.

Algorithm

1. Calculate the entropy of the target class in the training data.
2. Calculate information gain for every available attribute and select the attribute with the highest gain.
3. Create a decision node for that attribute and split the records by its categorical values.
4. Repeat ID3 on each subset using the remaining attributes; stop when a subset is pure or no attributes remain.
5. Follow the learned branches to classify the new sample; use the node majority for an unseen branch.

Entropy(S) = $-\Sigma\, p_i \log_2 p_i$; Gain(S, A) = Entropy(S) - weighted entropy after splitting on A.

Program:

id3_c.py - Python 3

```python
from collections import Counter
from math import log2
import pandas as pd

def entropy(labels):
    c = Counter(labels); n = len(labels)
    return -sum(x / n * log2(x / n) for x in c.values())

def gain(rows, attr, target):
    n = len(rows); base = entropy([r[target] for r in rows]); rest = 0.0
    for v in {r[attr] for r in rows}:
        part = [r for r in rows if r[attr] == v]
        rest += len(part) / n * entropy([r[target] for r in part])
    return base - rest

def id3(rows, attrs, target):
    labels = {r[target] for r in rows}
    if len(labels) == 1: return {"label": next(iter(labels))}
    if not attrs: return {"label": Counter(r[target] for r in rows).most_common(1)[0][0]}
    scores = {a: gain(rows, a, target) for a in attrs}; best = max(attrs, key=scores.get)
    node = {"attribute": best, "default": Counter(r[target] for r in rows).most_common(1)[0][0], "branches": {}}
    for v in sorted({r[best] for r in rows}):
        part = [r for r in rows if r[best] == v]
        node["branches"][v] = id3(part, [a for a in attrs if a != best], target)
    return node

def predict(node, sample):
    if "label" in node: return node["label"]
    child = node["branches"].get(sample.get(node["attribute"]))
    return predict(child, sample) if child else node["default"]

def show_tree(node, indent=""):
    if "label" in node:
        print(indent + "-> " + node["label"]); return
    print(indent + node["attribute"])
    for value, child in sorted(node["branches"].items()):
        if "label" in child: print(indent + "  " + value + " -> " + child["label"])
        else:
            print(indent + "  " + value + " ->")
            show_tree(child, indent + "    ")

frame = pd.DataFrame([
    ['Salaried', 'Good', 'High', 'Yes'],
    ['Salaried', 'Good', 'Low', 'Yes'],
    ['Self', 'Good', 'High', 'Yes'],
    ['Self', 'Good', 'Low', 'No'],
    ['Salaried', 'Poor', 'High', 'No'],
    ['Self', 'Poor', 'Low', 'No'],
    ['Salaried', 'Poor', 'Low', 'No'],
    ['Self', 'Good', 'Medium', 'Yes'],
], columns=['Employment', 'Credit', 'Income', 'Approved'])
records = frame.to_dict(orient="records")
attributes = ['Employment', 'Credit', 'Income']
target = 'Approved'
sample = {
    'Employment': 'Self',
    'Credit': 'Poor',
    'Income': 'High',
}
tree = id3(records, attributes, target)
print("Training examples:", len(records))
print("Root attribute:", tree.get("attribute", tree.get("label")))
print("Decision tree:"); show_tree(tree)
print("New sample:", sample)
print("Prediction:", predict(tree, sample))
```

Input

Bank Loan data set: 8 illustrative examples, 3 attributes (Employment, Credit, Income) and target Approved (Yes / No), represented as a pandas DataFrame.

Employment	Credit	Income	Approved
Salaried	Good	High	Yes
Salaried	Good	Low	Yes
Self	Good	High	Yes
Self	Good	Low	No
Salaried	Poor	High	No
Self	Poor	Low	No
Salaried	Poor	Low	No
Self	Good	Medium	Yes

Output

Terminal - Output

```
$ python id3_c.py
Training examples: 8
Root attribute: Credit
Decision tree:
Credit
  Good ->
    Income
      High -> Yes
      Low ->
        Employment
          Salaried -> Yes
          Self -> No
      Medium -> Yes
  Poor -> No
New sample: {'Employment': 'Self', 'Credit': 'Poor', 'Income': 'High'}
Prediction: No
```

Result

The learned tree uses the available attributes to classify loan records. For the new application with poor credit, the tree predicts Approved = No. This is a small illustrative dataset, not a real lending rule.

Problem 4: Car Purchase Decision

Aim

To implement ID3 using dictionary-based records and classify a new car-purchase decision from categorical attributes.

Algorithm

1. Calculate the entropy of the target class in the training data.
2. Calculate information gain for every available attribute and select the attribute with the highest gain.
3. Create a decision node for that attribute and split the records by its categorical values.
4. Repeat ID3 on each subset using the remaining attributes; stop when a subset is pure or no attributes remain.
5. Follow the learned branches to classify the new sample; use the node majority for an unseen branch.

Entropy(S) = -Σ $p_i \log_2 p_i$; Gain(S, A) = Entropy(S) - weighted entropy after splitting on A.

Program:

id3_d.py - Python 3

```python
from collections import Counter
from math import log2


def entropy(labels):
    c = Counter(labels); n = len(labels)
    return -sum(x / n * log2(x / n) for x in c.values())

def gain(rows, attr, target):
    n = len(rows); base = entropy([r[target] for r in rows]); rest = 0.0
    for v in {r[attr] for r in rows}:
        part = [r for r in rows if r[attr] == v]
        rest += len(part) / n * entropy([r[target] for r in part])
    return base - rest

def id3(rows, attrs, target):
    labels = {r[target] for r in rows}
    if len(labels) == 1: return {"label": next(iter(labels))}
    if not attrs: return {"label": Counter(r[target] for r in rows).most_common(1)[0][0]}
    scores = {a: gain(rows, a, target) for a in attrs}; best = max(attrs, key=scores.get)
    node = {"attribute": best, "default": Counter(r[target] for r in rows).most_common(1)[0][0], "branches": {}}
    for v in sorted({r[best] for r in rows}):
        part = [r for r in rows if r[best] == v]
        node["branches"][v] = id3(part, [a for a in attrs if a != best], target)
    return node

def predict(node, sample):
    if "label" in node: return node["label"]
    child = node["branches"].get(sample.get(node["attribute"]))
    return predict(child, sample) if child else node["default"]

def show_tree(node, indent=""):
    if "label" in node:
        print(indent + "-> " + node["label"]); return
    print(indent + node["attribute"])
    for value, child in sorted(node["branches"].items()):
        if "label" in child: print(indent + "  " + value + " -> " + child["label"])
        else:
            print(indent + "  " + value + " ->")
            show_tree(child, indent + "    ")

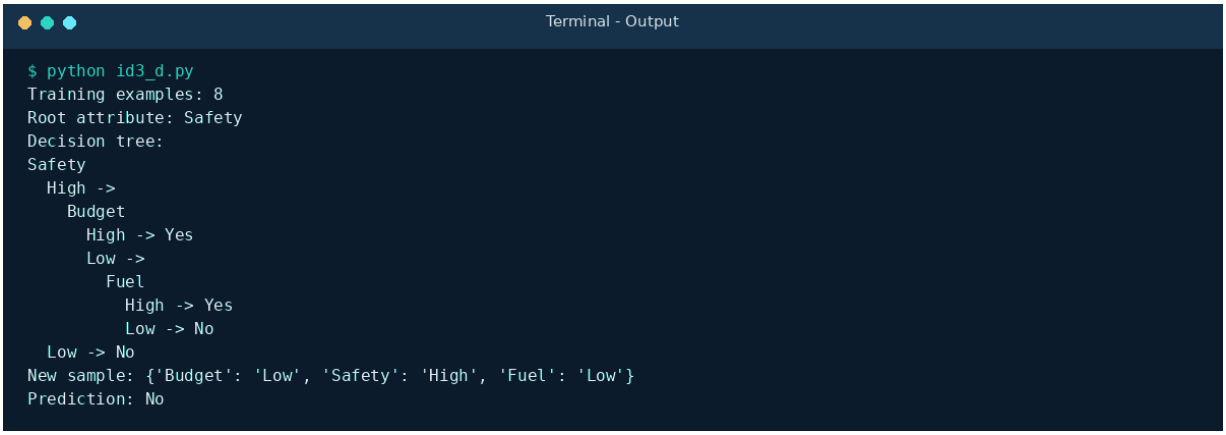
records = [
    {'Budget': 'High', 'Safety': 'High', 'Fuel': 'High', 'Buy': 'Yes'},
    {'Budget': 'High', 'Safety': 'High', 'Fuel': 'Low', 'Buy': 'Yes'},
    {'Budget': 'Low', 'Safety': 'High', 'Fuel': 'High', 'Buy': 'Yes'},
    {'Budget': 'Low', 'Safety': 'High', 'Fuel': 'Low', 'Buy': 'No'},
    {'Budget': 'High', 'Safety': 'Low', 'Fuel': 'High', 'Buy': 'No'},
    {'Budget': 'High', 'Safety': 'Low', 'Fuel': 'Low', 'Buy': 'No'},
    {'Budget': 'Low', 'Safety': 'Low', 'Fuel': 'High', 'Buy': 'No'},
    {'Budget': 'Low', 'Safety': 'Low', 'Fuel': 'Low', 'Buy': 'No'},
]
attributes = ['Budget', 'Safety', 'Fuel']
target = 'Buy'
sample = {
    'Budget': 'Low',
    'Safety': 'High',
    'Fuel': 'Low',
}
tree = id3(records, attributes, target)
print("Training examples:", len(records))
print("Root attribute:", tree.get("attribute", tree.get("label")))
print("Decision tree:"); show_tree(tree)
print("New sample:", sample)
print("Prediction:", predict(tree, sample))
```

Input

Car Purchase data set: 8 illustrative examples, 3 attributes (Budget, Safety, Fuel) and target Buy (Yes / No). Each row is represented as a dictionary.

Budget	Safety	Fuel	Buy
High	High	High	Yes
High	High	Low	Yes
Low	High	High	Yes
Low	High	Low	No
High	Low	High	No
High	Low	Low	No
Low	Low	High	No
Low	Low	Low	No

Output



Result

Safety is the most informative root attribute in this illustrative data. The sample with Low budget, High safety, and Low fuel rating follows the learned tree to Buy = No.

Problem 5: Outdoor Activity

Aim

To implement ID3 with NumPy categorical arrays, display the learned tree, and classify a new outdoor-activity sample.

Algorithm

1. Calculate the entropy of the target class in the training data.
2. Calculate information gain for every available attribute and select the attribute with the highest gain.
3. Create a decision node for that attribute and split the records by its categorical values.
4. Repeat ID3 on each subset using the remaining attributes; stop when a subset is pure or no attributes remain.
5. Follow the learned branches to classify the new sample; use the node majority for an unseen branch.

Entropy(S) = -Σ $p_i \log_2 p_i$; Gain(S, A) = Entropy(S) - weighted entropy after splitting on A.

Program:

id3_e.py - Python 3

```python
from collections import Counter
from math import log2
import numpy as np

def entropy(labels):
    c = Counter(labels); n = len(labels)
    return -sum(x / n * log2(x / n) for x in c.values())

def gain(rows, attr, target):
    n = len(rows); base = entropy([r[target] for r in rows]); rest = 0.0
    for v in {r[attr] for r in rows}:
        part = [r for r in rows if r[attr] == v]
        rest += len(part) / n * entropy([r[target] for r in part])
    return base - rest

def id3(rows, attrs, target):
    labels = {r[target] for r in rows}
    if len(labels) == 1: return {"label": next(iter(labels))}
    if not attrs: return {"label": Counter(r[target] for r in rows).most_common(1)[0][0]}
    scores = {a: gain(rows, a, target) for a in attrs}; best = max(attrs, key=scores.get)
    node = {"attribute": best, "default": Counter(r[target] for r in rows).most_common(1)[0][0], "branches": {}}
    for v in sorted({r[best] for r in rows}):
        part = [r for r in rows if r[best] == v]
        node["branches"][v] = id3(part, [a for a in attrs if a != best], target)
    return node

def predict(node, sample):
    if "label" in node: return node["label"]
    child = node["branches"].get(sample.get(node["attribute"]))
    return predict(child, sample) if child else node["default"]

def show_tree(node, indent=""):
    if "label" in node:
        print(indent + "-> " + node["label"]); return
    print(indent + node["attribute"])
    for value, child in sorted(node["branches"].items()):
        if "label" in child: print(indent + "  " + value + " -> " + child["label"])
        else:
            print(indent + "  " + value + " ->")
            show_tree(child, indent + "    ")

headers = ['Outlook', 'Temperature', 'Wind', 'Activity']
values = np.array([
    ['Sunny', 'Warm', 'Weak', 'Yes'],
    ['Sunny', 'Cool', 'Weak', 'Yes'],
    ['Sunny', 'Cool', 'Strong', 'No'],
    ['Rainy', 'Warm', 'Weak', 'No'],
    ['Rainy', 'Cool', 'Weak', 'No'],
    ['Cloudy', 'Warm', 'Weak', 'Yes'],
    ['Cloudy', 'Cool', 'Strong', 'Yes'],
    ['Cloudy', 'Warm', 'Strong', 'Yes'],
    ['Sunny', 'Warm', 'Strong', 'No'],
    ['Rainy', 'Warm', 'Strong', 'No'],
], dtype=str)
records = [dict(zip(headers, row.tolist())) for row in values]
attributes = ['Outlook', 'Temperature', 'Wind']
target = 'Activity'
sample = {
    'Outlook': 'Cloudy',
    'Temperature': 'Cool',
    'Wind': 'Strong',
}
tree = id3(records, attributes, target)
print("Training examples:", len(records))
print("Root attribute:", tree.get("attribute", tree.get("label")))
print("Decision tree:"); show_tree(tree)
print("New sample:", sample)
print("Prediction:", predict(tree, sample))
```

Input

Outdoor Activity data set: 10 illustrative examples, 3 attributes (Outlook, Temperature, Wind) and target Activity (Yes / No), stored in a NumPy string array.

Outlook	Temperature	Wind	Activity
Sunny	Warm	Weak	Yes
Sunny	Cool	Weak	Yes
Sunny	Cool	Strong	No
Rainy	Warm	Weak	No
Rainy	Cool	Weak	No
Cloudy	Warm	Weak	Yes
Cloudy	Cool	Strong	Yes
Cloudy	Warm	Strong	Yes
Sunny	Warm	Strong	No
Rainy	Warm	Strong	No

Output

Terminal - Output

```
$ python id3_e.py
Training examples: 10
Root attribute: Outlook
Decision tree:
Outlook
  Cloudy -> Yes
  Rainy -> No
  Sunny ->
    Wind
      Strong -> No
      Weak -> Yes
New sample: {'Outlook': 'Cloudy', 'Temperature': 'Cool', 'Wind': 'Strong'}
Prediction: Yes
```

Result

The tree is built from the NumPy array after converting rows to categorical records. The new sample has Outlook = Cloudy, which leads to Activity = Yes in this illustrative training set.